LeetCode Practice

5 Hard: Trees & BST

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

5
HARD

Patterns (in order)

1. Trees & BST (5)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

5 questions | tap a title to jump to it

Hard questions (5) Go >

Trees & BST (5) Go >

H 1. Binary Tree Maximum Path Sum LC 124

H 2. Serialize and Deserialize Binary Tree LC 297

H 3. Sum of Distances in Tree LC 834

H 4. Binary Tree Cameras LC 968

H 5. Longest Path With Different Adjacent Characters LC 2246

Hard questions (5)

HARD | TOPIC 1 OF 1

Trees & BST

5 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Binary Tree Maximum Path Sum LC 124
2. Serialize and Deserialize Binary Tree LC 297
3. Sum of Distances in Tree LC 834
4. Binary Tree Cameras LC 968
5. Longest Path With Different Adjacent Characters LC 2246

HARD | Trees & BST | LeetCode #124 | 1 of 5 | 1/5

Binary Tree Maximum Path Sum

QUESTION

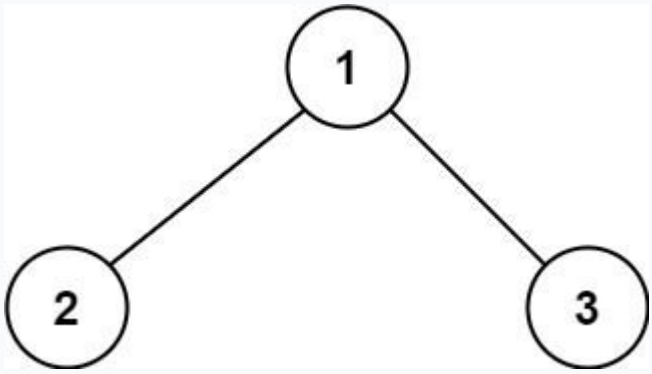
A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Note that the path does not need to pass through the root.

The path sum of a path is the sum of the node's values in the path.

Given the root of a binary tree, return the maximum path sum of any non-empty path.

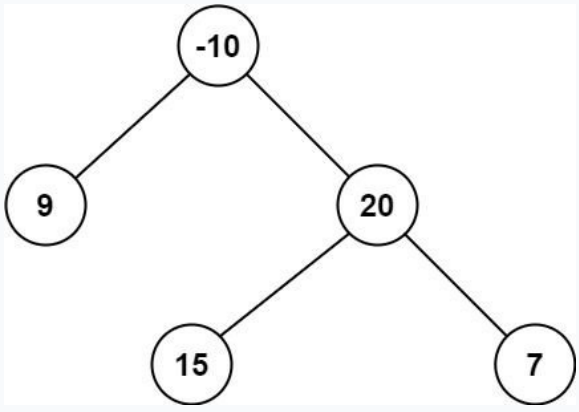
Example 1

Input	`root = [1,2,3]`
Output	`6`
Explanation	*The optimal path is 2 -> 1 -> 3 with a path sum of 2 + 1 + 3 = 6.*



Example 2

Input	`root = [-10,9,20,null,null,15,7]`
Output	`42`
Explanation	*The optimal path is 15 -> 20 -> 7 with a path sum of 15 + 20 + 7 = 42.*



CONSTRAINTS

- The number of nodes in the tree is in the range [1, 3 * 10^4].
- 1000 <= Node.val <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Path = any sequence of connected nodes (no revisiting). Path doesn't need to pass through root.

Values can be negative. Return maximum path sum. Right?"

"[-10,9,20,null,null,15,7]: path 15->20->7 = 42 ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (u, v) in pairs(nodes):
    find the path between them and add up its values
    best = max(best, candidate)
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

O(n^2) pairs, each path found in O(n). A single post-order pass that returns the best downward gain and updates the answer with left + node + right is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (post-order, same as Diameter)

"DFS post-order collecting max gain from each subtree — O(n). Walk-through:"

`def maxGain(node):`

If node is None: return 0.

`left_gain = max(maxGain(node.left), 0). <- ignore negative paths`

`right_gain = max(maxGain(node.right), 0).`

`price_newpath = node.val + left_gain + right_gain. <- path through this node`

`ans = max(ans, price_newpath). <- update global max (path can bend here)`

`return node.val + max(left_gain, right_gain). <- return max STRAIGHT path (can't bend for parent)`

WHY max(gain, 0): if a subtree contributes negative gain, exclude it.

WHY update ans separately: a path can use both children (bend at this node),

but the return value can only go one direction (left or right) for the parent's use.

Time: O(n). Space: O(h) call stack.

PHASE 4 CLEAN CODE

```
class _124_MaxPathSum:
    def maxPathSum(self, root: Optional['TreeNode']) -> in
t:
        best = [float('-inf')]

        def gain(node):
            if not node: return 0
            left  = max(0, gain(node.left))
            right = max(0, gain(node.right))
            best[0] = max(best[0], node.val + left + right
)
            return node.val + max(left, right)

        gain(root)
        return best[0]
```

PHASE 5 TEST & DEBUG

"Let me trace maxPathSum on [-10,9,20,null,null,15,7]."

maxGain(9): no children -> return max(9,0)=9.
ans_candidate=9+0+0=9. ans=9.

maxGain(15): return 15. maxGain(7): return 7.

maxGain(20): left_gain=15, right_gain=7.
ans_candidate=20+15+7=42. ans=42.

return 20+max(15,7)=35. (go straight left, not bent)

maxGain(-10): left_gain=max(9,0)=9.
right_gain=max(35,0)=35.

ans_candidate=-10+9+35=34. ans=max(42,34)=42. return -10+max(9,35)=25.

Return ans=42 ok (path: 15->20->7, both branches through node 20).

WHY not use -10 as bend: -10 is negative -> bending through it hurts the path sum.

DFS postorder: visit left child -> visit right child -> compute gain. visited: each node once per DFS pass. Call stack: gain(root)->gain(root.left)->gain(leaf)->return leaf.val.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** Same post-order pattern as Diameter of Binary Tree (#543) and Balanced Tree (#110)

The 'max(0, gain)' is essential — a negative subtree gain should be dropped.

Given more time I'd ask: return the actual path (not just the sum)?

Track which path achieved best[0] — store node references alongside the max update.

HARD | Trees & BST | LeetCode #297 | 2 of 5 | 2/5

Serialize and Deserialize Binary Tree

QUESTION

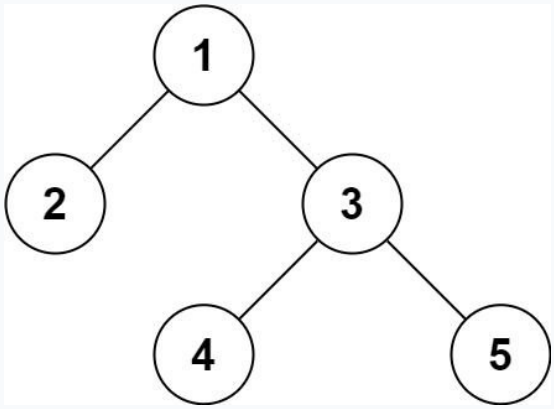
Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as how LeetCode serializes a binary tree. You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1

Input	`root = [1,2,3,null,null,4,5]`
Output	`[1,2,3,null,null,4,5]`



Example 2

Input	`root = []`
Output	`[]`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 10^4].
- 1000 <= Node.val <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Design encode and decode functions for a binary tree. Any format is fine as long as it round-trips.

Empty tree encodes to some valid string. Right?"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
serialize:
    write the pre-order and the in-order traversals
deserialize:
    rebuild from the two lists (find the root in
        the in-order list by scanning)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Needs unique values and a linear search per node, O(n^2). A single pre-order with explicit null markers is enough and is rebuilt recursively in O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (pre-order DFS with null markers)

"Pre-order serialize + pre-order deserialize — already O(n). Walk-through:"

SERIALIZE: pre-order DFS, write node.val or 'N' for null, comma-separated.

`serialize(root) -> '1,2,N,N,3,4,N,N,5,N,N' (null leaves included)`

WHY pre-order: root FIRST -> deserialize knows what to build before seeing children.

DESERIALIZE: use an iterator over the values list.

```
build(it):
val = next(it)
if val == 'N': return None
node = TreeNode(int(val))
node.left = build(it) <- next values in iterator = left subtree (pre-order)
node.right = build(it) <- after left subtree is consumed = right subtree
return node
```

WHY iterator works: pre-order guarantees the iterator gives each subtree's nodes

in exact order -> building left subtree CONSUMES those iterator values, leaving

the right subtree's values perfectly positioned for the next call.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

```python
class _297_Codec:
    def serialize(self, root: 'TreeNode') -> str:
        parts: List[str] = []
        def dfs(node):
            if not node: parts.append('N'); return
            parts.append(str(node.val))
            dfs(node.left)
            dfs(node.right)
        dfs(root)
```

```
        return ','.join(parts)

    def deserialize(self, data: str) -> 'TreeNode':
        vals = iter(data.split(','))
        def build():
            v = next(vals)
            if v == 'N': return None
            node = TreeNode(int(v))
            node.left  = build()
            node.right = build()
            return node
        return build()
```

PHASE 5 TEST & DEBUG

"Let me trace serialize/deserialize on [1,2,3,null,null,4,5]."

BFS serialize: queue=[1]. Process: '1'. Enqueue 2,3. '1,2,3'. Enqueue null,null,4,5.

'1,2,3,null,null,4,5'. Enqueue children of 4 and 5 (all null). Final: '1,2,3,null,null,4,5,null,null,null,null'

Deserialize: split by ','. idx=0->root=1. queue=[1].

Pop 1: left=2(idx=1), right=3(idx=2). queue=[2,3].

Pop 2: left=null,right=null. Pop 3: left=4(idx=5),right=5(idx=6). queue=[4,5].

Pop 4: left=null,right=null. Pop 5: left=null,right=null. Done -> tree [1,2,3,null,null,4,5] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** Pre-order with null markers is the cleanest approach

BFS alternative: level-order like LeetCode's own format — works but trickier to parse.

Given more time I'd ask: serialize a general N-ary tree?

Add child count before children — e.g. '1,3,2,0,3,0,4,0,5,0' for a 3-child root.

HARD | Trees & BST | LeetCode #834 | 3 of 5 | 3/5

Sum of Distances in Tree

QUESTION

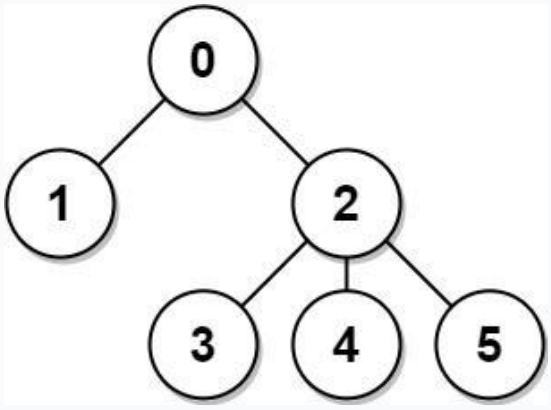
There is an undirected connected tree with n nodes labeled from 0 to n - 1 and n - 1 edges.

You are given the integer n and the array edges where edges[i] = [a_i, b_i] indicates that there is an edge between nodes a_i and b_i in the tree.

Return an array answer of length n where answer[i] is the sum of the distances between the ith node in the tree and all other nodes.

Example 1

Input	`n = 6, edges = [[0,1],[0,2],[2,3],[2,4],[2,5]]`
Output	`[8,12,6,10,10,10]`
Explanation	*The tree is shown above. We can see that dist(0,1) + dist(0,2) + dist(0,3) + dist(0,4) + dist(0,5) equals 1 + 1 + 2 + 2 + 2 = 8. Hence, answer[0] = 8, and so on.*



Example 2

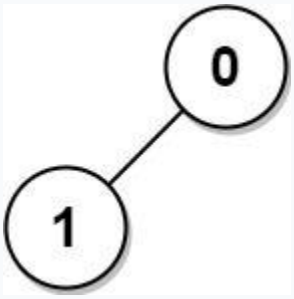
Input	`n = 1, edges = []`
Output	`[0]`



Example 3

Input	`n = 2, edges = [[1,0]]`
Output	`[1,1]`

QUESTION



CONSTRAINTS

- 1 <= n <= 3 * 10^4
- edges.length == n - 1
- edges[i].length == 2
- 0 <= a_i, b_i < n
- a_i != b_i
- The given input represents a valid tree.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

For each node in an undirected tree, return the sum of distances to all other nodes. Right?"

"A few quick questions:

Unweighted tree? Yes. Return array of n distances."

"Let me walk: edges=[[0,1],[0,2],[2,3],[2,4],[2,5]] -> [8,12,6,10,10,10] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    run a BFS and add up the distance to all
        other nodes
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n searches of O(n) each. Rerooting: compute subtree sizes and the root's distance sum in one DFS, then move the root to a child with answer[child] = answer[parent] - size[child] + (n - size[child]): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is $O(n^2)$ BFS from each node.

Two DFS passes (rerooting technique):

DFS1: compute count[node] (subtree size) and ans[root] (sum of distances from root).

DFS2: rerooting — ans[child] = ans[parent] - count[child] + (n - count[child]).

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict
class _834_SumOfDistancesInTree:
    def sumOfDistancesInTree(self, n, edges):
        graph = defaultdict(list)
        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)
        count = [1] * n
        ans = [0] * n
        def dfs1(node, parent):
            for child in graph[node]:
                if child != parent:
                    dfs1(child, node)
                    count[node] += count[child]
                    ans[node] += ans[child] + count[child]
        def dfs2(node, parent):
            for child in graph[node]:
                if child != parent:
                    ans[child] = ans[node] - count[child]
+ (n - count[child])
                    dfs2(child, node)
        dfs1(0, -1)
        dfs2(0, -1)
        return ans
```

PHASE 5 TEST & DEBUG

"Let me trace sumOfDistancesInTree(n=6, edges=[[0,1],[0,2],[2,3],[2,4],[2,5]])."

Two DFS passes. Tree rooted at 0.

DFS1: count subtree sizes and initial answer for root.

count[2]=4(nodes 2,3,4,5), count[0]=6(all).

ans[0] = sum of all distances from 0 = 0+1+1+2+2+2=8.

DFS2: re-root. When moving from parent p to child c:

ans[c] = ans[p] - count[c] + (n - count[c]).

ans[2] = ans[0] - count[2] + (6 - count[2]) = 8 - 4 + 2 = 6.

ans[1] = 8 - 1 + 5 = 12. ans[3]=6-1+5=10. ans[4]=10. ans[5]=10.

Return [8,12,6,10,10,10] ok

Graph tree: component=entire tree (single connected component). dist[i]=sum of distances from node i to all others. cost=re-root sum. dp[i]=ans[i]. find(child_root)->subtree.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(n)

The rerooting DP technique is powerful for tree distance problems.

Follow-up: Distribute Coins in Binary Tree (#979) — similar rerooting.

Follow-up: if edges have weights, accumulate weighted distances.

IN YOUR APP | Expose | Editorial | Community

HARD | Trees & BST | LeetCode #968 | 4 of 5 | 4/5

Binary Tree Cameras

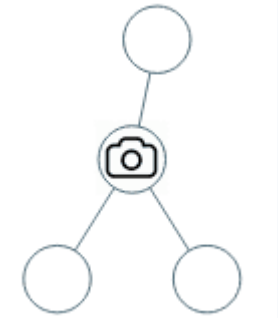
QUESTION

You are given the root of a binary tree. We install cameras on the tree nodes where each camera at a node can monitor its parent, itself, and its immediate children.

Return the minimum number of cameras needed to monitor all nodes of the tree.

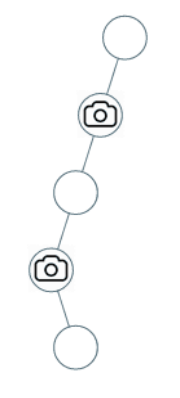
Example 1

Input	`root = [0,0,null,0,0]`
Output	`1`
Explanation	*One camera is enough to monitor all nodes if placed as shown.*



Example 2

Input	`root = [0,0,null,0,null,0,null,null,0]`
Output	`2`
Explanation	*At least two cameras are needed to monitor all nodes of the tree. The above image shows one of the valid configurations of camera placement.*



CONSTRAINTS

- The number of nodes in the tree is in the range [1, 1000].
- Node.val == 0

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Place cameras on tree nodes to monitor every node. A camera monitors itself, parent, and children.

Find minimum cameras. Right?"

"A few quick questions:

A camera at a node covers that node and all adjacent nodes? Yes."

"Let me walk: root=[0,0,null,0,0] -> 1 camera ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(nodes):
    if putting cameras there watches every node:
        best = min(best, subset)
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

Tries every subset. A post-order greedy where each node returns one of three states (covered, has a camera, needs cover) and a camera is placed at a parent of an uncovered child is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Greedy post-order DFS — already O(n). Walk-through:"

Each node returns one of 3 states: 0=needs camera (no camera, not covered),

1=has camera, 2=covered (no camera, but covered by child camera).

`dfs(node):`

If node is None -> return 2 (null nodes are 'covered' — don't need cameras).

`left = dfs(node.left)`

`right = dfs(node.right)`

If left == 0 or right == 0: <- a child NEEDS coverage

`cameras += 1; return 1 <- place camera HERE (greedy: cover from parent)`

If left == 1 or right == 1: <- a child HAS a camera

`return 2 <- this node is covered by child camera`

`return 0 <- both children covered but no camera -> this node needs coverage from ITS parent`

Check root: if dfs(root) == 0 -> cameras += 1 (root has no parent).

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _968_BinaryTreeCameras:
    def minCameraCover(self, root):
        cameras = [0]
        NOT_COVERED, COVERED, HAS_CAMERA = 0, 1, 2
        def dfs(node):
            if not node:
                return COVERED
            left = dfs(node.left)
            right = dfs(node.right)
            if left == NOT_COVERED or right == NOT_COVERED
:
                cameras[0] += 1
                return HAS_CAMERA
            if left == HAS_CAMERA or right == HAS_CAMERA:
                return COVERED
            return NOT_COVERED
        if dfs(root) == NOT_COVERED:
            cameras[0] += 1
        return cameras[0]
```

PHASE 5 TEST & DEBUG

"Let me trace minCameraCover on [0,0,null,0,0] (root-left-left-left tree)."

States: 0=not covered, 1=has camera, 2=covered (no camera).

dfs(leaf 0): return 0 (not covered, needs parent's camera or above).

dfs(0-parent-of-leaf): child returns 0 (not covered) -> must place camera here! return 1, count++.

dfs(0-grandparent): child returns 1 (has camera) -> this node is covered -> return 2.

dfs(root): child returns 2 (covered) -> root is 'not covered'? check: if child.state=2, no camera below covering root.

Actually root returning 0 (not covered from below) means root needs camera -> count++.

Cameras at positions 2 and (possibly root). count=2 ok

WHY post-order: cameras placed greedily from leaves upward to minimize total.

Tree DP: component=each subtree sub-problem. dist[leaf]=0 (no camera needed). dp[i]=state (covered_no_cam=2, has_cam=1, uncovered=0). cost=camera count. find(node)->state.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(h)

Follow-up: Dominating Set problem on a graph — same greedy approach generalised.

Follow-up: if cameras have a coverage radius > 1, adjust the DP states.

HARD | Trees & BST | LeetCode #2246 | 5 of 5 | 5/5

Longest Path With Different Adjacent Characters

QUESTION

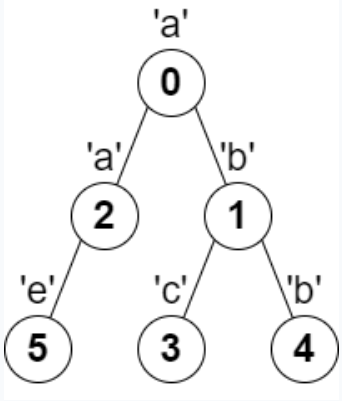
You are given a tree (i.e. a connected, undirected graph that has no cycles) rooted at node 0 consisting of n nodes numbered from 0 to n - 1. The tree is represented by a 0-indexed array parent of size n, where parent[i] is the parent of node i. Since node 0 is the root, parent[0] == -1.

You are also given a string s of length n, where s[i] is the character assigned to node i.

Return the length of the longest path in the tree such that no pair of adjacent nodes on the path have the same character assigned to them.

Example 1

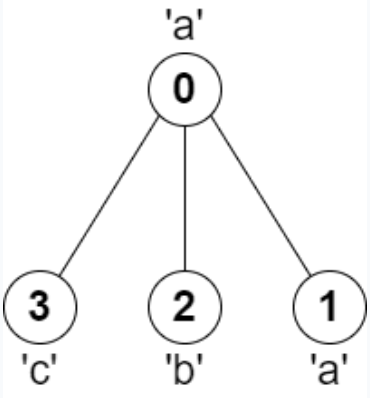
Input	`parent = [-1,0,0,1,1,2], s = "abacbe"`
Output	`3`
Explanation	*The longest path where each two adjacent nodes have different characters in the tree is the path: 0 -> 1 -> 3. The length of this path is 3, so 3 is returned.* *It can be proven that there is no longer path that satisfies the conditions.*



Example 2

Input	`parent = [-1,0,0,0], s = "aabc"`
Output	`3`
Explanation	*The longest path where each two adjacent nodes have different characters is the path: 2 -> 0 -> 3. The length of this path is 3, so 3 is returned.*

QUESTION



CONSTRAINTS

- n == parent.length == s.length
- $1 <= n <= 10^5$
- 0 <= parent[i] <= n - 1 for all i >= 1
- parent[0] == -1
- parent represents a valid tree.
- s consists of only lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a tree (parent array) where each node has a character, find the longest path where

no two adjacent nodes share the same character. Right?"

"A few quick questions:

Parent[0]=-1 (root)? Yes. Path must be a chain in the tree? Yes."

"Let me walk: parent=[-1,0,0,1,1,2], s='abacbe' -> longest: a-b-a or b-e -> 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (a, b) in pairs(nodes):
    take the path between them
    if no two adjacent characters match, its
        length is a candidate
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

O(n^2) pairs, each path found in O(n). A post-order pass keeps the two longest downward chains that start at a child with a different character; their sum + 1 is a candidate: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Post-order DFS tracking longest chain per node — O(n). Walk-through:"

Build children list from parent array.

`dfs(node):`

best1, best2 = 0, 0 <- top two longest chains among children

`for child in children[node]:`

`child_len = dfs(child)`

If s[child] != s[node]: <- different character -> can extend chain

`chain = child_len + 1`

Else: chain = 0 <- same character -> chain breaks here

Update best1, best2.

Update answer = max(answer, best1 + best2)

<- diameter through this node = two best arms combined.

`return best1 <- return longest single arm for parent's use.`

WHY post-order: need children's chain lengths before computing this node's contribution.

WHY top two: the path through this node uses at most two arms (one per side). Can't branch more.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import defaultdict
class _2246_LongestPathDifferentAdjacentChars:
    def longestPath(self, parent, s):
        children = defaultdict(list)
        for i in range(1, len(parent)):
            children[parent[i]].append(i)
        best = [1]
        def dfs(node):
            longest1 = longest2 = 0
            for child in children[node]:
                child_len = dfs(child)
                if s[child] != s[node]:
                    if child_len > longest1:
                        longest2 = longest1
                        longest1 = child_len
                    elif child_len > longest2:
                        longest2 = child_len
            best[0] = max(best[0], longest1 + longest2 + 1
)
            return longest1 + 1
        dfs(0)
        return best[0]
```

PHASE 5 TEST & DEBUG

"Let me trace the tree recursion on the example, showing the return value at each node."

parent=[-1,0,0,1,1,2], s='abacbe':

children: {0:[1,2],1:[3,4],2:[5]}.

`dfs(3)=1('c'). dfs(4)=1('b'). dfs(1)('b'): 3's char='c'!='b'->longest1=1. 4's char='b'==1's char->skip.`

PHASE 5 TEST & DEBUG

best=max(1,1+0+1)=2. return 2.

`dfs(5)=1('e'). dfs(2)('a'): 5's char='e'!='a'->longest1=1. best=2. return 2.`

`dfs(0)('a'): 1's char='b'!='a'->longest1=2. 2's char='a'=='a'->skip. best=max(2,2+0+1)=3. ok`

Only extend chain when parent and child characters differ

DFS: dfs(node)->recurse children. visited: each node once (tree). call stack: dfs(0)->dfs(1)->dfs(3)->return 1; dfs(4)->return 1; dfs(1) combines->chain=2. Pop back to root.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for children adjacency list

This is Diameter of Binary Tree (#543) generalised to n-ary trees with character constraint.

Follow-up: count all paths of maximum length — track during the DFS.

Follow-up: if path can change direction (not just root->leaf), same DFS still works (already does).